

Assignment – 9.5

Name : P. Niharika

Ht.no : 2303A51694

Batch : 24

Problem 1: String Utilities Function

Consider the following Python function:

```
def reverse_string(text):
```

```
    return text[::-1]
```

Task:

1. Write documentation in: o (a) Docstring o (b) Inline comments o (c) Google-style documentation
2. Compare the three documentation styles.
3. Recommend the most suitable style for a utility-based string library.

```
Docexample.py > reverse_string
1  def reverse_string(text):
2      """
3      Reverses the given string.
4
5      Parameters:
6      text (str): The string to be reversed.
7
8      Returns:
9      str: The reversed string.
10     """
11     return text[::-1]
12 # (b) Inline comments
13 def reverse_string(text):
14     # Take the input string 'text'
15     # Use slicing to reverse the string
16     # Return the reversed string
17     return text[::-1]
18 # (c) Google style documentation
19 def reverse_string(text):
20     """
21     Reverses the given string.
22
23     Args:
24     | text (str): Input string to reverse.
25
26     Returns:
27     | str: Reversed version of the input string.
28     """
29     return text[::-1]
```

```

PS C:\Users\2303a\OneDrive\Desktop\AI> python -m pydoc Docexample
Help on module Docexample:

NAME
    Docexample

FUNCTIONS
    reverse_string(text)
        Reverses the given string.

        Args:
            text (str): Input string to reverse.

        Returns:
            str: Reversed version of the input string.
-- More --

```

Problem 2: Password Strength Checker

Consider the function:

```

def check_strength(password):
    return len(password) >= 8

```

Task:

1. Document the function using docstring, inline comments, and Google style.
2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

```

Docexample.py > ...
1  def check_strength(password):
2      """
3      Checks whether the password is at least 8 characters long.
4
5      Parameters:
6      password (str): The password to check.
7
8      Returns:
9      bool: True if password length is >= 8, else False.
10     """
11     return len(password) >= 8
12
13  # (b) Inline comments
14  def check_strength(password):
15      # Check if password length is greater than or equal to 8
16      return len(password) >= 8
17
18  # (c) Google style documentation
19  def check_strength(password):
20      """
21      Checks the strength of a password.
22
23      Args:
24      password (str): Password string.
25
26      Returns:
27      bool: True if password length is 8 or more, otherwise False.
28      """
29      return len(password) >= 8

```

```

PS C:\Users\2303a\OneDrive\Desktop\AI> C:\Users\2303a\AppData\Local\Microsoft\Windows\Python\python.exe C:\Users\2303a\OneDrive\Desktop\AI\Docexample.py
PS C:\Users\2303a\OneDrive\Desktop\AI> python -m pydoc Docexample
Help on module Docexample:

NAME
    Docexample

FUNCTIONS
    check_strength(password)
        Checks the strength of a password.

NAME
    Docexample

FUNCTIONS
    check_strength(password)
        Checks the strength of a password.

FUNCTIONS
    check_strength(password)
        Checks the strength of a password.

    check_strength(password)
        Checks the strength of a password.

    check_strength(password)
        Checks the strength of a password.

```

Problem 3: Math Utilities Module

Task:

1. Create a module `math_utils.py` with functions:
 - o `square(n)`
 - o `cube(n)`
 - o `factorial(n)`
2. Generate docstrings automatically using AI tools.
3. Export documentation as an HTML file.

```
math_utils.py > ...
1  def square(x):
2      """
3      Returns the square of a number.
4      parameter x: The number to be squared.
5      return: The square of x.
6      int or float: The number to be squared.
7      """
8      return x * x
9  def cube(x):
10     """
11     Returns the cube of a number.
12     parameter x: The number to be cubed.
13     return: The cube of x.
14     int or float: The number to be cubed.
15     """
16     return x * x * x
17  def factorial(n):
18     """
19     Returns the factorial of a number.
20     parameter n: The number to compute the factorial of.
21     return: The factorial of n.
22     """
23     if n == 0:
24         return 1
25     else:
26         return n * factorial(n - 1)
27  print(square.__doc__)
28  print(cube.__doc__)
29  print(factorial.__doc__)
```

```
PS C:\Users\2303a\OneDrive\Desktop\AI> python -m pydoc math_utils.py
```

```
Returns the square of a number.  
parameter x: The number to be squared.  
return: The square of x.  
int or float: The number to be squared.
```

```
Returns the cube of a number.  
parameter x: The number to be cubed.  
return: The cube of x.  
int or float: The number to be cubed.
```

```
Returns the factorial of a number.  
parameter n: The number to compute the factorial of.  
return: The factorial of n.
```

```
No Python documentation found for 'math_utils.py'.  
Use help() to get the interactive help utility.  
Returns the cube of a number.  
parameter x: The number to be cubed.  
return: The cube of x.  
int or float: The number to be cubed.
```

```
Returns the factorial of a number.  
parameter n: The number to compute the factorial of.  
return: The factorial of n.
```

```
No Python documentation found for 'math_utils.py'.  
Use help() to get the interactive help utility.  
int or float: The number to be cubed.
```

```
Returns the factorial of a number.  
parameter n: The number to compute the factorial of.
```

Problem 4: Attendance Management Module

Task:

1. Create a module attendance.py with functions:
 - o mark_present(student)
 - o mark_absent(student)
 - o get_attendance(student)
2. Add proper docstrings.
3. Generate and view documentation in terminal and browse

```
get_attendance > get_attendance
1  attendance = {}
2  def mark_present(student):
3      """
4      Marks a student as present in the attendance record.
5      Parameters:
6      student (str): The name of the student to be marked as present.
7      """
8      attendance[student] = "Present"
9  def mark_absent(student):
10     """
11     Marks a student as absent in the attendance record.
12     Parameters:
13     student (str): The name of the student to be marked as absent.
14     """
15     attendance[student] = "Absent"
16  def get_attendance(student):
17     """
18     Returns the attendance status of a student.
19     Parameters:
20     student (str): The name of the student whose attendance is to be retrieved.
21     Returns:
22     str: The attendance status of the student.
23     """
24     return attendance.get(student, "Not Found")
```

```
PS C:\Users\2303a\OneDrive\Desktop\AI> python -m pydoc -p 1234
Server ready at http://localhost:1234/
Server commands: [b]rowser, [q]uit
server> █
```



```
math_util

# def square(n) :
#     """Returns the square of a number.
#     demonstrates how to use docstrings in Python.
#     Parameters:
#     n (int): The number to be squared.
#     Returns:int: The square of n.
#     """
#     return n * n
# def cube(n) :
#     """Returns the cube of a number.
#     demonstrates how to use docstrings in Python.
#     Parameters:
#     n (int): The number to be cubed.
#     Returns:int: The cube of n,
#     """
#     return n * n * n
# def factorial(n) :
#     """Returns the factorial of a number.
#     demonstrates how to use docstrings in Python.
#     Parameters:
#     n (int): The number to calculate the factorial of.
#     Returns:int: The factorial of n.
#     """
#     if n == 0: # check if n is 0 and return 1 if it is because factorial of 0 is 1
#         return 1 # factorial of 0 is defined to be 1
#     else:
#         return n * factorial(n - 1) # Recursive call to calculate factorial of n
# print(square.__doc__)
# print(cube.__doc__)
# print(factorial.__doc__)

Functions

get_attendance(student)
    Returns the attendance status of a student.
    Parameters:
    student (str): The name of the student whose attendance is to be retrieved.
    Returns:
    str: The attendance status of the student.

mark_absent(student)
    Marks a student as absent in the attendance record.
    Parameters:
    student (str): The name of the student to be marked as absent.

mark_present(student)
    Marks a student as present in the attendance record.
    Parameters:
    student (str): The name of the student to be marked as present.

Data

attendance = {}
```

Problem 5:

File Handling Function

Consider the function: def

read_file(filename): with

open(filename, 'r') as f:

return f.read()

Task:

1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation.

```

math_utils.py > read_file
1  # DocString style:
2  def read_file(filename):
3      """
4      Reads the content of a file and returns it as a string.
5      Parameters:
6      filename (str): The name of the file to be read.
7      Returns:
8      str: The content of the file.
9      Raises:
10     FileNotFoundError: If the specified file does not exist.
11     IOError: If an I/O error occurs while reading the file.
12     """
13     try:
14         with open(filename, 'r') as f:
15             return f.read()
16     except FileNotFoundError:
17         print(f"Error: The file '{filename}' was not found.")
18         raise
19     except IOError as e:
20         print(f"An I/O error occurred: {e}")
21         raise
22 # Google Style Docstring:
23 def read_file(filename):
24     """
25     Reads the content of a file and returns it as a string.
26     Args:
27     filename (str): The name of the file to be read.
28     Returns:
29     str: The content of the file.
30     Raises:
31     FileNotFoundError: If the specified file does not exist.
32     IOError: If an I/O error occurs while reading the file.
33     """
34     try:
35         with open(filename, 'r') as f:
36             return f.read()
37     except FileNotFoundError:

```



```

36         return f.read()
37     except FileNotFoundError:
38         print(f"Error: The file '{filename}' was not found.")
39         raise
40     except IOError as e:
41         print(f"An I/O error occurred: {e}")
42         raise
43 #python style docstring:
44 def read_file(filename):
45     """
46     Reads the content of a file and returns it as a string.
47     :param filename: The name of the file to be read.
48     :type filename: str
49     :return: The content of the file.
50     :rtype: str
51     :raises FileNotFoundError: If the specified file does not exist.
52     :raises IOError: If an I/O error occurs while reading the file.
53     """
54     try:
55         with open(filename, 'r') as f:
56             return f.read()
57     except FileNotFoundError:
58         print(f"Error: The file '{filename}' was not found.")
59         raise
60     except IOError as e:
61         print(f"An I/O error occurred: {e}")
62         raise

```

Use `help(str)` for help on the `str` class.

```
PS C:\Users\2303a\OneDrive\Desktop\AI> python -m pydoc math_utils
```

Help on module `math_utils`:

NAME

`math_utils` - # DocString style:

FUNCTIONS

`read_file(filename)`

Reads the content of a file and returns it as a string.

:param filename: The name of the file to be read.

:type filename: str

:return: The content of the file.

:rtype: str

:raises FileNotFoundError: If the specified file does not exist.

:raises IOError: If an I/O error occurs while reading the file.

FILE

`c:\users\2303a\onedrive\desktop\ai\math_utils.py`